1 Benchmark Modeling Details

This appendix provides additional details on how the benchmark families used in our evaluation arise from software-engineering analysis and synthesis tasks. For each family, we first describe the original problem at the level of programs, transition systems, rewrite systems, circuits, or arithmetic constructions. We then present the corresponding MaxSMT(NIA) or OMT(NIA) encoding, including the variables, constraints, and objective function.

The purpose of this appendix is to make clear that these benchmarks are not arbitrary collections of non-linear integer constraints. They are generated from structured reasoning tasks in software engineering, such as termination analysis, non-termination analysis, equivalence checking, and arithmetic synthesis. The integer variables in the formulas typically represent coefficients of invariants or ranking functions, parameters of algebraic interpretations, entries of matrices, witnesses for non-termination, relational parameters for equivalence checking, or construction choices in arithmetic synthesis.

1.1 VeryMax MaxSMT(NIA) Families: CInteger and ITS

Original problem. The CInteger and ITS families originate from termination-oriented program analysis. In the CInteger family, the input is a C program over integer variables. The program is first translated into an integer transition system. In the ITS family, the input is already an integer transition system. Therefore, both families can be described uniformly at the transition-system level.

Let

$$\mathcal{T} = (L, \ell_0, X, T)$$

be an integer transition system, where L is the set of control locations, ℓ_0 is the initial location, $X = \{x_1, \ldots, x_n\}$ is the set of integer program variables, and T is the set of transitions. Each transition $\tau \in T$ is written as

$$\tau = (\ell, \rho_\tau(X, X'), \ell'),$$

where ℓ and ℓ' are the source and target locations, and $\rho_\tau(X, X')$ is the transition relation over pre-state variables X and post-state variables X'.

The original program-analysis problem is to synthesize a proof object showing that the program makes progress towards termination. Such a proof object consists of location invariants and ranking functions. A typical invariant template at location ℓ is

$$I_\ell(X) \equiv a_{\ell,0} + \sum_{i=1}^{n} a_{\ell,i} x_i \geq 0,$$

and a typical ranking-function template is

$$R(X) = r_0 + \sum_{i=1}^{n} r_i x_i.$$

The unknown coefficients $a_{\ell,i}$ and r_i are integer parameters to be synthesized.

The proof object must be sound with respect to the program semantics. In particular, the invariant must hold at initial states and must be preserved by transitions. In addition, for each transition, the analyzer would like to establish ranking-related progress conditions, such as boundedness, strict decrease, and non-increase. However, under a fixed template, it may be impossible to establish all progress obligations simultaneously. Therefore, the original analysis task is naturally formulated as a weighted optimization problem: synthesize a sound proof object and maximize the total weight of progress obligations that it can establish.

MaxSMT(NIA) encoding. The encoding introduces integer variables for template coefficients and auxiliary variables generated by the arithmetic encoding. Let Θ denote all unknown template coefficients and let Y denote auxiliary integer variables.

The first class of hard constraints encodes invariant initiation:

$$C_{\text{init}}(\Theta, Y) = \mathsf{Enc}\left(\forall X.\ \Theta_0(X) \Rightarrow I_{\ell_0}(X)\right),$$

where $\Theta_0(X)$ denotes the initial-state condition.

The second class of hard constraints encodes invariant preservation. For each transition $\tau = (\ell, \rho_\tau, \ell')$, we have

$$C_\tau(\Theta, Y) = \mathsf{Enc}\left(\forall X, X'.\ I_\ell(X) \wedge \rho_\tau(X, X') \Rightarrow I_{\ell'}(X')\right).$$

The progress obligations are encoded separately. For each transition $\tau = (\ell, \rho_\tau, \ell')$, the boundedness obligation is

$$B_\tau(\Theta, Y) = \mathsf{Enc}\left(\forall X, X'.\ I_\ell(X) \wedge \rho_\tau(X, X') \Rightarrow R(X) \geq 0\right).$$

The strict-decrease obligation is

$$S_\tau(\Theta, Y) = \mathsf{Enc}\left(\forall X, X'.\ I_\ell(X) \wedge \rho_\tau(X, X') \Rightarrow R(X) - R(X') \geq 1\right).$$

The non-increase obligation is

$$N_\tau(\Theta, Y) = \mathsf{Enc}\left(\forall X, X'.\ I_\ell(X) \wedge \rho_\tau(X, X') \Rightarrow R(X) - R(X') \geq 0\right).$$

For each progress obligation, the MaxSMT encoding introduces a Boolean relaxation variable. For transition τ, these variables are

$$p_\tau^B, \quad p_\tau^S, \quad p_\tau^N.$$

The guarded hard clauses are

$$p_\tau^B \vee B_\tau(\Theta, Y),$$
$$p_\tau^S \vee S_\tau(\Theta, Y),$$

and

$$p_\tau^N \vee N_\tau(\Theta, Y).$$

If p_τ^S is assigned true, for example, then the strict-decrease obligation for transition τ is relaxed.

Let

$$Q_\tau^B = B_\tau, \quad Q_\tau^S = S_\tau, \quad Q_\tau^N = N_\tau.$$

The full set of hard clauses is

$$\mathcal{H} = \{C_{\text{init}}\} \cup \{C_\tau \mid \tau \in T\} \cup \{p_\tau^q \vee Q_\tau^q \mid \tau \in T,\ q \in \{B, S, N\}\}.$$

The soft clauses penalize relaxed progress obligations:

$$\mathcal{S} = \{(\neg p_\tau^q, \omega_\tau^q) \mid \tau \in T,\ q \in \{B, S, N\}\},$$

where $\omega_\tau^q > 0$ is the weight of obligation q on transition τ.

Thus, the complete MaxSMT(NIA) problem is

$$\min_{\Theta, Y, p} \sum_{\tau \in T} \sum_{q \in \{B,S,N\}} \omega_\tau^q \cdot \mathbf{1}\{p_\tau^q = \text{true}\}$$

subject to

$$C_{\text{init}}(\Theta, Y),$$
$$\bigwedge_{\tau \in T} C_\tau(\Theta, Y),$$

and

$$\bigwedge_{\tau \in T} \bigwedge_{q \in \{B,S,N\}} \left(p_\tau^q \vee Q_\tau^q(\Theta, Y)\right).$$

Equivalently, minimizing the MaxSMT cost maximizes the total weight of progress obligations established by the synthesized proof object. A zero-cost solution establishes all encoded progress obligations, while a positive-cost solution still satisfies all soundness constraints but leaves some progress obligations unproved. In the CInteger family, the transition system is obtained from C integer programs. In the ITS family, the transition system is given directly.

1.2 AProVE OMT(NIA) Family

Original problem. The AProVE family originates from automated termination analysis. The original problem is to prove termination of a program or rewrite system by synthesizing an algebraic termination certificate. A common form of such a certificate is a polynomial interpretation.

Let $\mathcal{R}$ be a set of rewrite rules. Each rule has the form

$$l \to r.$$

A polynomial interpretation assigns each function symbol f of arity k a polynomial over its arguments:

$$[f]_\Theta(x_1, \ldots, x_k) = c_f + \sum_{i=1}^{k} a_{f,i} x_i + \sum_{1 \le i \le j \le k} b_{f,i,j} x_i x_j + \cdots,$$

where $c_f, a_{f,i}, b_{f,i,j}, \ldots$ are unknown integer coefficients. For a term t, its interpretation $[t]_\Theta$ is obtained recursively from the interpretations of the function symbols occurring in t.

The original termination problem asks whether there exists an interpretation under which every rewrite rule decreases. Formally, for each rule $l \to r$, the certificate must satisfy

$$\forall X.\ [l]_\Theta(X) > [r]_\Theta(X).$$

Over integer arithmetic, strict decrease is written as

$$\forall X.\ [l]_\Theta(X) - [r]_\Theta(X) \ge 1.$$

The interpretation also needs to satisfy non-negativity and monotonicity constraints, such as

$$c_f \ge 0, \quad a_{f,i} \ge 0, \quad b_{f,i,j} \ge 0, \quad \ldots$$

and, when strict monotonicity is required for an argument,

$$a_{f,i} \ge 1.$$

Thus, at the level of the original problem, a satisfying assignment gives concrete coefficients for a valid termination certificate.

OMT(NIA) encoding. The OMT encoding introduces integer variables for all interpretation coefficients and auxiliary variables. Let Θ be the set of interpretation coefficients and let Y be the set of auxiliary variables.

The monotonicity and non-negativity constraints are encoded as

$$\mathsf{Mono}_f(\Theta)$$

for each function symbol f. The orientation constraint for a rule $l \to r$ is encoded as

$$\mathsf{Orient}_{l,r}(\Theta, Y) = \mathsf{Enc}\left(\forall X.\ [l]_\Theta(X) - [r]_\Theta(X) \geq 1\right).$$

Additional constraints introduced by the termination encoding are denoted by

$$\mathsf{Aux}(\Theta, Y).$$

The hard formula is

$$\varphi_{\mathrm{AProVE}}(\Theta, Y) = \left(\bigwedge_f \mathsf{Mono}_f(\Theta)\right) \wedge \left(\bigwedge_{l \to r \in \mathcal{R}} \mathsf{Orient}_{l,r}(\Theta, Y)\right) \wedge \mathsf{Aux}(\Theta, Y).$$

The OMT objective optimizes a numerical parameter of the termination certificate. Let z be such a parameter, for example an interpretation coefficient or an auxiliary bound. The OMT(NIA) problem is

$$\min_{\Theta, Y} z$$

subject to

$$\varphi_{\mathrm{AProVE}}(\Theta, Y).$$

Equivalently,

$$\min_{\Theta, Y} z \quad \text{s.t.} \quad \left(\bigwedge_f \mathsf{Mono}_f(\Theta)\right) \wedge \left(\bigwedge_{l \to r \in \mathcal{R}} \mathsf{Orient}_{l,r}(\Theta, Y)\right) \wedge \mathsf{Aux}(\Theta, Y).$$

Thus, the OMT instance asks for a valid termination certificate with an optimized numerical parameter.

1.3 VeryMax-Derived OMT(NIA) Families: CInteger, ITS, and SAT14

Original problem. The VeryMax-derived OMT families come from the same type of program-analysis proof search as the VeryMax MaxSMT benchmarks. The original problem is to synthesize invariants, ranking functions, or related transition arguments for C programs or integer transition systems.

Let

$$\mathcal{T} = (L, \ell_0, X, T)$$

be the transition system under analysis. The proof object contains invariant templates

$$I_\ell(X) \equiv a_{\ell,0} + \sum_{i=1}^{n} a_{\ell,i} x_i \geq 0$$

and ranking templates

$$R(X) = r_0 + \sum_{i=1}^{n} r_i x_i.$$

The original proof-search problem is to find coefficients such that the proof object is valid. At the program level, this means that the invariant holds initially, is preserved by all transitions, and supports the ranking or transition argument required by the analysis.

OMT(NIA) encoding. Let Θ denote the invariant and ranking-function coefficients, and let Y denote auxiliary variables. The hard constraints first encode invariant initiation:

$$C_{\mathrm{init}}(\Theta, Y) = \mathsf{Enc}\left(\forall X.\ \Theta_0(X) \Rightarrow I_{\ell_0}(X)\right).$$

For each transition $\tau = (\ell, \rho_\tau, \ell')$, the hard constraints encode invariant preservation:

$$C_\tau(\Theta, Y) = \mathsf{Enc}\left(\forall X, X'.\ I_\ell(X) \wedge \rho_\tau(X, X') \Rightarrow I_{\ell'}(X')\right).$$

The transition-progress constraints generated by the analysis are denoted by

$$P_\tau(\Theta, Y).$$

For example, P_τ may include ranking boundedness,

$$\mathsf{Enc}\left(\forall X, X'.\ I_\ell(X) \wedge \rho_\tau(X, X') \Rightarrow R(X) \geq 0\right),$$

strict decrease,

$$\mathsf{Enc}\left(\forall X, X'.\ I_\ell(X) \wedge \rho_\tau(X, X') \Rightarrow R(X) - R(X') \geq 1\right),$$

or other transition obligations used by the original analysis.

The full hard formula is

$$\varphi_{\mathrm{VM}}(\Theta, Y) = C_{\mathrm{init}}(\Theta, Y) \wedge \bigwedge_{\tau \in T} C_\tau(\Theta, Y) \wedge \bigwedge_{\tau \in T} P_\tau(\Theta, Y).$$

Let z be a numerical parameter of the proof object, such as a template coefficient, a bound, or an auxiliary witness value. The OMT(NIA) problem is

$$\min_{\Theta, Y} z$$

subject to

$$C_{\mathrm{init}}(\Theta, Y),$$
$$\bigwedge_{\tau \in T} C_\tau(\Theta, Y),$$

and

$$\bigwedge_{\tau \in T} P_\tau(\Theta, Y).$$

Equivalently,

$$\min_{\Theta, Y} z \quad \text{s.t.} \quad \varphi_{\mathrm{VM}}(\Theta, Y).$$

The CInteger family is derived from C integer programs, the ITS family is derived from integer transition systems, and the SAT14 family consists of VeryMax-generated formulas included in SMT-LIB. In all cases, the hard formula encodes the validity of a program-analysis proof object, while the objective optimizes a numerical component of that object.

1.4 LassoRanker and Ultimate OMT(NIA) Families

Original problem. The LassoRanker and Ultimate families are derived from the analysis of lasso-shaped programs. A lasso-shaped program consists of a finite stem followed by a loop:

$$\mathrm{STEM}(X_0, X) \quad \text{and} \quad \mathrm{LOOP}(X, X').$$

The stem describes how a loop head can be reached, and the loop describes one iteration of the recurrent part of the program.

The original problem is either to prove termination by synthesizing a ranking argument, or to prove non-termination by synthesizing a recurrent execution witness.

For termination, one searches for a ranking function

$$R(X) = r_0 + r^T X$$

and possibly a supporting invariant

$$I(X) \equiv s_0 + s^T X \geq 0.$$

The intended meaning is that I holds whenever the loop is entered, I is preserved by the loop, and R is bounded and strictly decreases along every loop iteration.

For non-termination, one searches for a witness showing the existence of an infinite execution. A common witness is a geometric non-termination argument. Suppose the loop relation is represented as

$$A \begin{pmatrix} X \\ X' \end{pmatrix} \leq b.$$

A simple geometric witness consists of an initial recurrent state x_1, a direction vector y, and a scalar λ. It represents the infinite sequence

$$x_t = x_1 + \sum_{i=0}^{t-1} \lambda^i y.$$

If the stem reaches x_1 and the loop relation is closed under this geometric progression, then the program has an infinite execution.

OMT(NIA) encoding. For termination instances, the encoding introduces ranking coefficients r_0, r, invariant coefficients s_0, s, and auxiliary variables Y. The hard constraints are as follows.

The reachability or initiation constraint is

$$C_{\mathrm{reach}} = \mathsf{Enc}\left(\forall X_0, X.\ \mathrm{STEM}(X_0, X) \Rightarrow I(X)\right).$$

The invariant-preservation constraint is

$$C_{\mathrm{inv}} = \mathsf{Enc}\left(\forall X, X'.\ I(X) \wedge \mathrm{LOOP}(X, X') \Rightarrow I(X')\right).$$

The boundedness constraint is

$$C_{\mathrm{bound}} = \mathsf{Enc}\left(\forall X, X'.\ I(X) \wedge \mathrm{LOOP}(X, X') \Rightarrow R(X) \geq 0\right).$$

The strict-decrease constraint is

$$C_{\mathrm{dec}} = \mathsf{Enc}\left(\forall X, X'.\ I(X) \wedge \mathrm{LOOP}(X, X') \Rightarrow R(X) - R(X') \geq 1\right).$$

Thus, the termination formula is

$$\varphi_{\text{term}} = C_{\text{reach}} \wedge C_{\text{inv}} \wedge C_{\text{bound}} \wedge C_{\text{dec}} \wedge \text{Aux}.$$

For non-termination instances, the encoding introduces variables x_0, x_1, y, λ and auxiliary variables Y. The hard constraints include

$$C_{\text{stem}} = \text{STEM}(x_0, x_1),$$

$$C_{\text{first}} = A \begin{pmatrix} x_1 \\ x_1 + y \end{pmatrix} \leq b,$$

$$C_{\text{closed}} = A \begin{pmatrix} y \\ \lambda y \end{pmatrix} \leq 0,$$

and

$$C_{\lambda} = (\lambda > 0).$$

The product terms λy_i in C_{closed} introduce non-linear integer arithmetic. The non-termination formula is

$$\varphi_{\text{nonterm}} = C_{\text{stem}} \wedge C_{\text{first}} \wedge C_{\text{closed}} \wedge C_{\lambda} \wedge \text{Aux}.$$

Depending on the instance, the hard formula is either

$$\varphi_{\text{lasso}} = \varphi_{\text{term}}$$

or

$$\varphi_{\text{lasso}} = \varphi_{\text{nonterm}}.$$

Let z be a numerical parameter of the synthesized ranking argument or non-termination witness. The OMT(NIA) problem is

$$\min z \quad \text{s.t.} \quad \varphi_{\text{lasso}}.$$

For termination instances, optimizing z searches for a quantitatively smaller ranking argument or invariant parameter. For non-termination instances, optimizing z searches for an improved numerical parameter of the recurrent witness.

1.5 calypto OMT(NIA) Family

Original problem. The calypto family originates from industrial sequential equivalence checking. The original problem is to compare two sequential designs and prove that their observable behaviors are equivalent. Let the two systems be

$$M_1 = (S_1, I_1, T_1, O_1) \quad \text{and} \quad M_2 = (S_2, I_2, T_2, O_2),$$

where S_i are state variables, I_i are initial-state predicates, T_i are transition relations, and O_i are output functions.

The semantic equivalence problem asks whether the two systems produce the same observable output behavior for all input traces. This can be written abstractly as

$$\forall U.\, O_1^*(U) = O_2^*(U),$$

where $O_i^*(U)$ denotes the output trace of system M_i under input trace U.

In practice, the proof is often established by synthesizing a relational invariant or state correspondence

$$R_\Theta(S_1, S_2),$$

which relates states of the two systems. In non-cycle-accurate equivalence checking, the two systems may not align cycle by cycle, so the proof may also involve alignment parameters.

OMT(NIA) encoding. The encoding introduces variables Θ for the relational template and variables Y for auxiliary witnesses or alignment parameters. The initial consistency constraint is

$$C_{\text{init}} = \text{Enc}\left(\forall S_1, S_2.\, I_1(S_1) \wedge I_2(S_2) \Rightarrow R_\Theta(S_1, S_2)\right).$$

For cycle-accurate equivalence, the relational preservation constraint is

$$C_{\text{step}} = \text{Enc}\left(\forall S_1, S_2, U, S_1', S_2'.\, R_\Theta(S_1, S_2) \wedge T_1(S_1, U, S_1') \wedge T_2(S_2, U, S_2') \Rightarrow R_\Theta(S_1', S_2')\right).$$

The output-equivalence constraint is

$$C_{\text{out}} = \text{Enc}\left(\forall S_1, S_2, U.\, R_\Theta(S_1, S_2) \Rightarrow O_1(S_1, U) = O_2(S_2, U)\right).$$

For non-cycle-accurate equivalence checking, suppose the two systems are compared after k_1 and k_2 transition steps. Then the composed transition relations are

$$T_1^{k_1}(S_1, U, S_1') \quad \text{and} \quad T_2^{k_2}(S_2, U, S_2').$$

The preservation constraint becomes

$$C_{\text{step}}^{k_1,k_2} = \text{Enc}\left(\forall S_1, S_2, U, S_1', S_2'.\, R_\Theta(S_1, S_2) \wedge T_1^{k_1}(S_1, U, S_1') \wedge T_2^{k_2}(S_2, U, S_2') \Rightarrow R_\Theta(S_1', S_2')\right),$$

and the output-equivalence constraint becomes

$$C_{\text{out}}^{k_1,k_2} = \mathsf{Enc}\left(\forall S_1, S_2, U.\ R_\Theta(S_1, S_2) \Rightarrow O_1^{k_1}(S_1, U) = O_2^{k_2}(S_2, U)\right).$$

The hard formula is schematically

$$\varphi_{\text{calypto}}(\Theta, Y) = C_{\text{init}} \wedge C_{\text{step}}^{k_1,k_2} \wedge C_{\text{out}}^{k_1,k_2} \wedge \mathsf{Aux}(\Theta, Y).$$

Let z be a numerical parameter of the relation, alignment, or auxiliary witness. The OMT(NIA) problem is

$$\min_{\Theta,Y} z \quad \text{s.t.} \quad \varphi_{\text{calypto}}(\Theta, Y).$$

Thus, the OMT instance searches for a valid equivalence witness with an optimized quantitative parameter while preserving all hard equivalence constraints.

1.6 leipzig OMT(NIA) Family

Original problem. The leipzig family comes from termination analysis using matrix interpretations for term rewriting. The original problem is to synthesize a matrix interpretation proving that every rewrite rule decreases under a well-founded order.

Let $\mathcal{R}$ be a rewrite system. For a function symbol f of arity k, a matrix interpretation assigns an affine transformation

$$[f]_\Theta(\vec{x}_1, \ldots, \vec{x}_k) = F_{f,1}\vec{x}_1 + \cdots + F_{f,k}\vec{x}_k + \vec{f},$$

where $F_{f,i}$ are matrices of unknown integer coefficients and $\vec{f}$ is a vector of unknown integer constants.

The original termination problem asks whether there exist such matrices and vectors so that every rule $l \to r$ decreases. The interpretation must also be monotone and non-negative.

OMT(NIA) encoding. The encoding introduces matrix-entry variables

$$F_{f,i}[p, q]$$

and vector-entry variables

$$\vec{f}[p].$$

The non-negativity constraints are

$$C_{\text{nonneg}} = \left(\bigwedge_{f,i,p,q} F_{f,i}[p, q] \geq 0\right) \wedge \left(\bigwedge_{f,p} \vec{f}[p] \geq 0\right).$$

For each rewrite rule $l \to r$, the orientation constraint requires that $[l]_\Theta$ is greater than $[r]_\Theta$. A componentwise encoding can be written as

$$C_{l,r}^{\text{strict}} = \mathsf{Enc}\left(\forall X.\ ([l]_\Theta(X))_1 - ([r]_\Theta(X))_1 \geq 1\right),$$

for a strict component, and

$$C_{l,r}^{\text{weak}}(j) = \mathsf{Enc}\left(\forall X.\ ([l]_\Theta(X))_j - ([r]_\Theta(X))_j \geq 0\right)$$

for the remaining components j.

The orientation formula for rule $l \to r$ is

$$\mathsf{Orient}_{l,r}(\Theta, Y) = C_{l,r}^{\text{strict}} \wedge \bigwedge_{j \neq 1} C_{l,r}^{\text{weak}}(j).$$

Nested terms introduce non-linear constraints. For example, if $t = f(g(x))$, then

$$[t]_\Theta = F_f(F_g x + \vec{g}) + \vec{f} = (F_f F_g)x + F_f\vec{g} + \vec{f}.$$

The product $F_f F_g$ contains entries

$$\sum_h F_f[p, h] \cdot F_g[h, q],$$

which are non-linear monomials over integer unknowns.

The full hard formula is

$$\varphi_{\text{leipzig}}(\Theta, Y) = C_{\text{nonneg}} \wedge \left(\bigwedge_{l \to r \in \mathcal{R}} \mathsf{Orient}_{l,r}(\Theta, Y)\right) \wedge \mathsf{Aux}(\Theta, Y).$$

Let z be a matrix entry, vector entry, or auxiliary parameter. The OMT(NIA) problem is

$$\min_{\Theta,Y} z \quad \text{s.t.} \quad \varphi_{\text{leipzig}}(\Theta, Y).$$

Thus, the OMT instance searches for a valid matrix interpretation with an optimized numerical component.

1.7 mcm OMT(NIA) Family

Original problem. The mcm family comes from the Multiple Constant Multiplication problem. Given a set of constants

$$C = \{c_1, \ldots, c_m\},$$

the original synthesis problem is to implement all multiplications

$$c_1 \cdot x, \ldots, c_m \cdot x$$

using shifts, additions, and subtractions. Since shifts are inexpensive in hardware, the main synthesis goal is to use as few addition and subtraction operations as possible.

Let $R_0 = \{1\}$ be the set of constants initially available, corresponding to the input x. After one addition or subtraction step, new constants can be generated as

$$|2^{s_1}u + \sigma 2^{s_2}v|,$$

where u, v are previously available constants, $s_1, s_2 \in \mathbb{N}$ are shift amounts, and $\sigma \in \{-1, 1\}$. Define recursively

$$R_k = R_{k-1} \cup \{|2^{s_1}u + \sigma 2^{s_2}v| \mid u, v \in R_{k-1},\ s_1, s_2 \in \mathbb{N},\ \sigma \in \{-1, 1\}\}.$$

The original MCM optimization problem is

$$K^* = \min\{K \mid C \subseteq R_K\}.$$

OMT(NIA) encoding. A bounded encoding fixes a construction length K and asks whether all target constants can be generated. Let

$$v_0 = 1$$

and let $v_1, \ldots, v_K$ be generated intermediate constants. For each step k, selection variables indicate which previous constants, shifts, and sign are used. Let

$$y_{k,i,j,s_1,s_2,\sigma} \in \{0, 1\}$$

denote that step k uses operands v_i and v_j, shifts s_1 and s_2, and sign σ.

The exactly-one constraint for step k is

$$C^k_{\text{one}} = \left(\sum_{i,j,s_1,s_2,\sigma} y_{k,i,j,s_1,s_2,\sigma} = 1 \right).$$

The generation constraint is

$$C^k_{\text{gen}} = \bigwedge_{i,j,s_1,s_2,\sigma} \left(y_{k,i,j,s_1,s_2,\sigma} = 1 \Rightarrow v_k = \left|2^{s_1}v_i + \sigma 2^{s_2}v_j\right| \right).$$

The absolute value can be encoded using a disjunction:

$$v_k = 2^{s_1}v_i + \sigma 2^{s_2}v_j \quad \vee \quad v_k = -\left(2^{s_1}v_i + \sigma 2^{s_2}v_j\right).$$

The target coverage constraint requires every target constant to be generated:

$$C_{\text{cover}} = \bigwedge_{c \in C} \bigvee_{0 \le k \le K} v_k = c.$$

The hard formula for a fixed construction length K is

$$\varphi^K_{\text{mcm}}(V, Y) = (v_0 = 1) \wedge \left(\bigwedge_{k=1}^{K} C^k_{\text{one}} \right) \wedge \left(\bigwedge_{k=1}^{K} C^k_{\text{gen}} \right) \wedge C_{\text{cover}}.$$

Let z be a numerical parameter of the arithmetic construction, such as an intermediate constant or an auxiliary construction variable. The OMT(NIA) problem is

$$\min_{V,Y} z \quad \text{s.t.} \quad \varphi^K_{\text{mcm}}(V, Y).$$

The hard constraints ensure that the arithmetic construction realizes all target constants, while the objective selects a quantitatively preferable construction among feasible ones.